Python × tkinter でつくる
公式仕様テトリス完全攻略テキスト

中学生からプログラミングのプロへ！
図解満載＆カラーコードで見るゲームアルゴリズム

【ポイント】 本書の特徴と学習の進め方

★ 本書の設計思想：挫折ゼロの「2部構成カリキュラム」★

【第1部：MVP基本完成編（第1回～第6回）】
・画面の座標とお絵かきから始め、6回で「100%遊べる本格テトリス」を完成させます。
・途中で授業のコマ数が足りなくなったり、進度が遅れても、第6回で全員が完成の感動を体験できます！

【第2部：プロ仕様アップグレード編（第7回～第12回）】
・完成したテトリスに、世界の公式テトリスと同じ神機能を1つずつ追加します。
・落下予測の影（ゴースト）、7-bag公平乱数、ロックディレイ（遊び時間）、ホールド、SRS壁キック、T-Spin判定を完全攻略！

対象：Python初心者～中学生（図とカラーコードで直感理解）
環境：Python 3.x（外部ライブラリ不要・標準機能だけで完全自作）

【全12回の学習ロードマップ】

プロのゲーム開発と同じ「動く最小限を作り（第1部）、そこに神機能を追加する（第2部）」手順で進めます。

部	回	テーマ	身につくこと・ゴール	サンプルファイル
第1部 基本完成	第1回	tkinterとお絵かき	ウィンドウ作成、Canvas、単色ブロック描画、画面座標	lesson1_canvas.py
	第2回	マス目とミノの定義	四角形から10×20方眼紙へ！盤面リスト・ミノ7種・完成	lesson2_grid_blocks.py
	第3回	移動・落下・ループ	root.after()自動落下、←/→移動、↓ソフト、↑ハード	lesson3_movement.py
	第4回	当たり判定 can_place	壁・床・ブロックとの衝突判定（動かす前のお試し一歩）	lesson4_collision.py
	第5回	回転の仕組み（数学）	転置と反転の2ステップ魔法、A/Dキー回転、めり込み防止	lesson5_rotation.py
	第6回	★基本テトリス完成！	盤面固定、ライン消去、ゲームオーバー【ここで完全遊べる】	lesson6_basic_tetris.py
第2部 プロ進化	第7回	ゴーストブロック（影）	落下予想位置をグレーの影でリアルタイム表示！	lesson7_ghost.py
	第8回	7-bagシステム	7種類が袋から均等に出る「7種1巡」の公平な仕組み	lesson8_bag.py
	第9回	ロックディレイ	底に着いても約0.5秒間操作できる遊び時間	lesson9_lock_delay.py
	第10回	ホールド＆NEXT表示	Spaceキーでキープ・交換（1手1回制限）、NEXT枠表示	lesson10_hold_next.py
	第11回	公式回転「SRS」	狭い隙間や壁際でもスッと回る壁キックテーブル	lesson11_srs.py
	第12回	T-Spin判定＆究極完成	3コーナー判定によるTスピン演出、究極完全版	lesson12_complete.py

【ポイント】 操作方法を最初から統一しよう！

【全編共通】公式テトリス準拠の統一操作キー一覧
・← / →：左右移動　|　↓：ソフトドロップ（下に1マス進む）
・↑：ハードドロップ（一瞬で底まで急降下して即固定！）
・A：左回転（反時計回り）　|　D：右回転（時計回り）
・Space：ホールド（手持ちを一時キープ・交換、第10回〜）
・P：ポーズ（一時停止）　|　R：リスタート（ゲームをやり直す）

第1回：tkinterの基本とお絵かき

【この回のゴール】 GUIウィンドウを立ち上げ、キャンバス上に単色の四角形を描画する



図解解説：画面の座標系は「左上が原点」＆「下に行くほどYが増える」！

数学のグラフと違い、コンピュータの画面は『左上端』が (0, 0) で、『下に行くほど Y が増える』ルールです。
テトリス盤面は横10マス×縦20マス。1マス20pxとすると、全体は「幅 200px × 高さ 400px」になります。

> **【ポイント】 なぜ root.mainloop() が必要なの？**
>
> Pythonは最後の行まで進むと自動終了してしまいます。
> root.mainloop()
> を呼ぶことで、「キー入力や閉じるボタンを押すまで画面を開いたまま待機する」見張り番がスタートします！

四角形を描くコードとつまずき防止



lesson1_canvas.py

```
import tkinter as tk

root = tk.Tk()
root.title("第1回：単色キャンバス")

# 幅200px, 高さ400pxの黒いキャンバス
canvas = tk.Canvas(root, width=200, height=400, bg="#12121c")
canvas.pack()

# マス目 (3, 4) に四角形を描く: (3*20, 4*20, 3*20+20, 4*20+20)
canvas.create_rectangle(60, 80, 80, 100, fill="#00d2d3", outline="#232334")

root.mainloop() # ★見張り番（これがないと一瞬で閉じる！）
```

【ポイント】「画面が出ない・一瞬で消える」ときのチェックリスト

① ファイル未保存（Ctrl + S 忘れ）： 空白ファイルが実行されています！
② 最後の root.mainloop() の書き忘れ： 見張り番がいないため即終了しています。

第2回：10×20の盤面とブロックの定義

【この回のゴール】 四角形1個から10×20マスへ！方眼紙グリッドとブロックの形を定義する



四角形1個（第1回）から10×20盤面への3ステップ進化！

第1回では create_rectangle で画面に四角形を1個だけ描きました。
しかしテトリスを作るには『画面全体に10×20マスの方眼紙を広げ、そのマス目番号に合わせてブロックを描く』必要があります！

・空きマスは None、ブロックがあるマスにはその単色カラー（'#00d2d3' 等）を書き込みます。
・リストの指定は field[y][x]（縦の行yが先、横の列xが後！） になるのが最大ポイントです！

盤面の定義

```
# 縦20行×横10列の盤面リスト（最初はすべて空っぽ None）
field = [[None for _ in range(COLUMNS)] for _ in range(ROWS)]

# 例: 5行目の3列目にブロックを置く
field[5][3] = "#a55eea" # field[y][x]（縦yが先、横xが後！）
```

【ポイント】 ピクセルから「マス目思考」へ！

画面の座標 (px, py) を直接いじるのではなく、内部データ field[y][x] で『どのマスに何があるか』を記憶・管理します！

四角形からグリッドへ！ステップ・バイ・ステップ進化



ステップ解説：どうやって四角形から10×20マスにするのか？

① 20pxごとに線を引いて方眼紙を作る：
for x in range(0, 200, 20): canvas.create_line(...) で縦線10本・横線20本を一瞬で引きます。
② マス目座標 (x, y) をピクセルに変換して描く：
第1回の create_rectangle を使い、px = x * 20, py = y * 20 を渡せば、狙ったマス目に四角形がカチッと収まります！

● ● ● グリッド線とマス目描画の関数

```
# ステップ1: 10x20のグリッド線を描く（20px間隔）
for x in range(0, 200, 20):
    canvas.create_line(x, 0, x, 400, fill="#232334")
for y in range(0, 400, 20):
    canvas.create_line(0, y, 200, y, fill="#232334")

# ステップ2: マス目座標 (x, y) をピクセルに換算して四角形を描く！
def draw_cell(x, y, color):
    px, py = x * 20, y * 20
    canvas.create_rectangle(px, py, px + 20, py + 20, fill=color, outline="#232334")
```

【ポイント】 『マス目番号 × 20』が魔法の合言葉！

ピクセルを1個ずつ計算する必要はありません！マス目番号に 20 をかけるだけで、画面の正確な位置に描けます。

格子状の 0 と 1 で描くテトリミノ7種



SHAPES と描画処理

```
# 0＝あきマス（透明） / 1＝ブロックがあるマス（色で塗りつぶす！）
SHAPES = {
    "T": [[0, 1, 0], [1, 1, 1], [0, 0, 0]],
    "I": [[0, 0, 0, 0], [1, 1, 1, 1], [0, 0, 0, 0], [0, 0, 0, 0]],
    "O": [[1, 1], [1, 1]],
    "S": [[0, 1, 1], [1, 1, 0], [0, 0, 0]], ...
}
# マス目 (start_x, start_y) にミノを描くコード
for r_idx in range(len(shape)):
    for c_idx in range(len(shape[r_idx])):
        if shape[r_idx][c_idx] == 1:
            px = (start_x + c_idx) * 20
            py = (start_y + r_idx) * 20
            canvas.create_rectangle(px, py, px + 20, py + 20, fill=color)
```

【ポイント】 どのミノも【1の数＝絶対に4個】（テトラ＝4）！

コードの 1 の数字を数えてみてください。どのミノもきっちり4個あります！1があるマスだけに四角形を描画します。

【完成形コード】lesson2_grid_blocks.py

第1回のお絵かきコードに、グリッド線描画とミノ描画、Spaceキー切り替えを合体させた完全なコードです！

lesson2_grid_blocks.py

```
import tkinter as tk
import random

root = tk.Tk()
root.title("第2回：マス目とブロックの定義")

CELL_SIZE = 20
COLUMNS, ROWS = 10, 20
CW, CH = CELL_SIZE * COLUMNS, CELL_SIZE * ROWS

canvas = tk.Canvas(root, width=CW, height=CH, bg="#12121c")
canvas.pack()

# 縦20行×横10列の盤面リスト（空マスは None）
field = [[None for _ in range(COLUMNS)] for _ in range(ROWS)]

# 7種類のミノ（0＝透明, 1＝ブロック）
SHAPES = {
    "T": [[0, 1, 0], [1, 1, 1], [0, 0, 0]],
    "I": [[0, 0, 0, 0], [1, 1, 1, 1], [0, 0, 0, 0], [0, 0, 0, 0]],
    "O": [[1, 1], [1, 1]],
    "S": [[0, 1, 1], [1, 1, 0], [0, 0, 0]],
    "Z": [[1, 1, 0], [0, 1, 1], [0, 0, 0]],
    "J": [[1, 0, 0], [1, 1, 1], [0, 0, 0]],
    "L": [[0, 0, 1], [1, 1, 1], [0, 0, 0]]
}
COLORS = {
    "I": "#00d2d3", "O": "#feca57", "T": "#a55eea",
    "S": "#1dd1a1", "Z": "#ff6b6b", "J": "#54a0ff", "L": "#ff9f43"
}
current_type = "T"

def draw():
    canvas.delete("all")
    # 1. 10x20のグリッド線を描く
    for x in range(0, CW, CELL_SIZE):
        canvas.create_line(x, 0, x, CH, fill="#232334")
    for y in range(0, CH, CELL_SIZE):
        canvas.create_line(0, y, CW, y, fill="#232334")

    # 2. ミノをマス目 (x=3, y=5) に描く
    shape = SHAPES[current_type]
    color = COLORS[current_type]
    start_x, start_y = 3, 5
    for r in range(len(shape)):
        for c in range(len(shape[r])):
            if shape[r][c] == 1:
                px = (start_x + c) * CELL_SIZE
                py = (start_y + r) * CELL_SIZE
                canvas.create_rectangle(px, py, px + CELL_SIZE, py + CELL_SIZE, fill=color, outline="#232334")

def on_space(event):
    global current_type
    current_type = random.choice(list(SHAPES.keys()))
    draw()

root.bind("<space>", on_space)
draw()
root.mainloop()
```

【ポイント】 実行時のチェックポイント＆動作確認

① 実行すると黒い画面に10×20のマス目と紫色のTミノが出現します！
② 【Spaceキー】を押すと、ランダムに別のミノ（I, O, S, Z, J, L）に切り替わります！
★ キーが反応しないときは、日本語入力（全角）ではなく「半角英数」になっているか確認しましょう。

【特別図解コラム】テトリミノの謎を解け！0と1の枠の秘密

「ブロックの実体は4個なのに、なぜTは3×3マスで、Iは4×4マスの枠なのか？」
中学生が100%疑問に思うこの鋭い謎を、幾何学と回転の視点からスッキリ解き明かします！



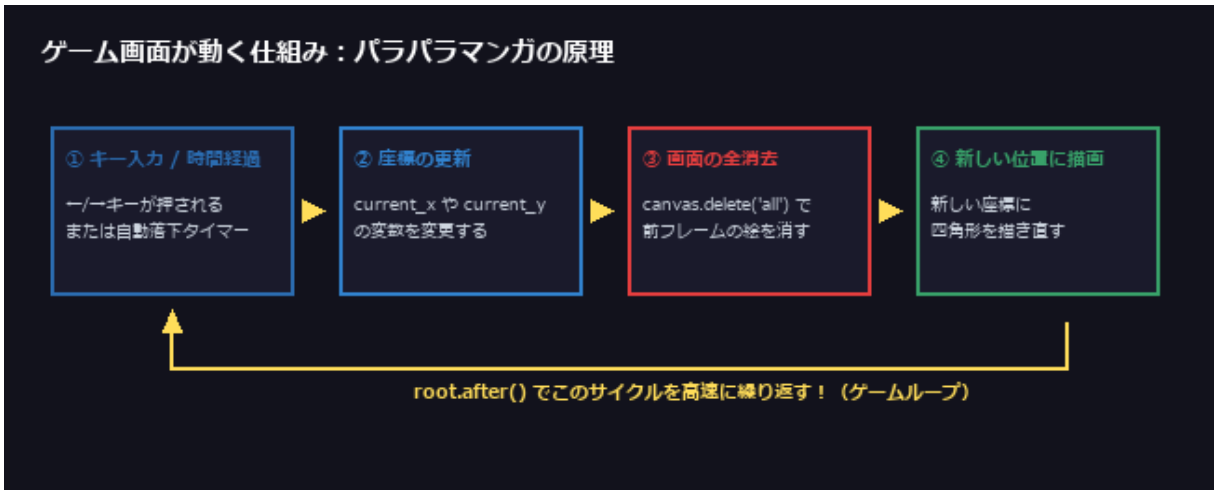
謎の答え：正方形の枠にするからこそ「回転軸」が定まる！

・T, J, L, S, Z（3×3マス）：奇数マスなので真ん中のマス [1][1]が中心軸になり、コマのようにその場で回せます。
・Iミノ（4×4マス）：棒の長さが4あるため3×3には入りません！偶数マスなので中央の十字交差点を軸に対称に回します。
・Oミノ（2×2マス）：正方形なので回しても形が変わらず、回転計算自体が不要です。

> **【ポイント】 画鋲（がびょう）のたとえで覚えよう！**
>
> Tミノはマス目のド真ん中に画鋲を刺してクルクル回すイメージ。Iミノはマスとマスの間の十字線に画鋲を刺して回すイメージです！

第3回：動かそう！移動・落下・ゲームループ

【この回のゴール】 アニメーションの仕組みを理解し、自動落下とキー入力を結びつける



パラパラマンガの原理 & なぜ time.sleep() はダメなのか？

ゲーム画面は「全消去 ➔ 座標更新 ➔ 再描画」の高速ループで動いています。

・time.sleep(0.5) を使うと、Python全体が眠り込んでキー操作を受け付けなくなります！

・代わりに root.after(500, auto_drop) を使い、「0.5秒後にまた呼んでね」と予約して即座に戻します。

自動落下のゲームループ

```
def auto_drop():
    global current_y
    current_y += 1 # 1マス下へ進める
    draw() # 画面再描画
    root.after(500, auto_drop) # ★0.5秒後にまた自分を呼ぶ！（予約）
```

キー操作の実装（移動・ソフト・ハードドロップ）



キー入力ハンドラ

```
def on_key_press(event):
    global current_x, current_y
    if event.keysym == "Left":
        current_x -= 1 # ← 左移動
    elif event.keysym == "Right":
        current_x += 1 # → 右移動
    elif event.keysym == "Down":
        current_y += 1 # ↓ ソフトドロップ（下に1マス進む）
    elif event.keysym == "Up":
        # ↑ ハードドロップ（底まで一気にワープして即固定！）
        while can_place(current_shape, current_x, current_y + 1):
            current_y += 1
        lock_piece()
    draw()

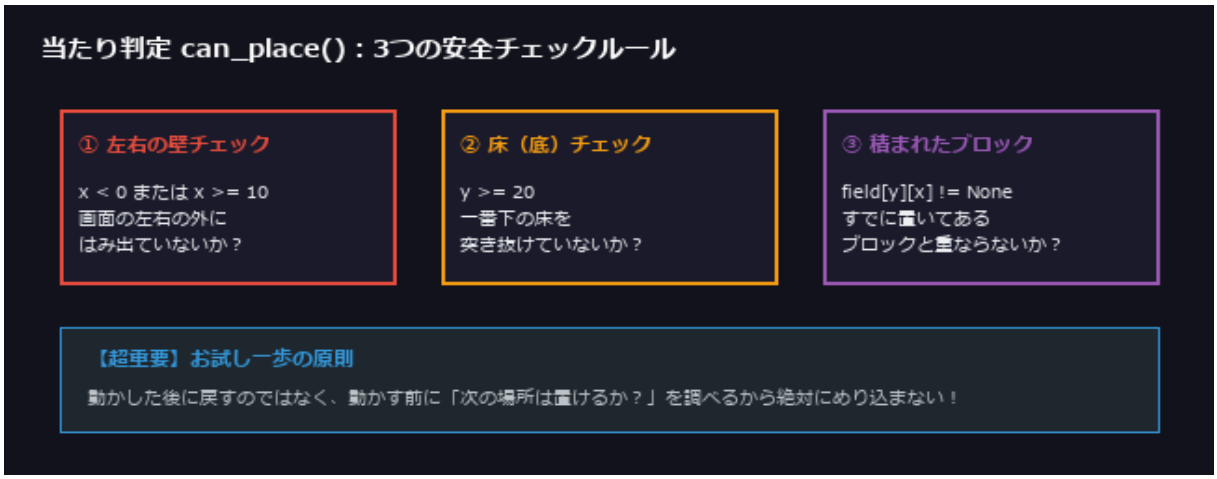
root.bind("<Key>", on_key_press)
```

【ポイント】 キーを押したら即 draw() でキビキビ動く！

キー操作を受け取った直後に draw() を呼ぶことで、次の自動落下を待たずに画面が即座に反応します！

第4回：絶対にすり抜けない！当たり判定 can_place

【この回のゴール】 壁・床・すでに置かれたブロックとの衝突を防ぐ関数を作る



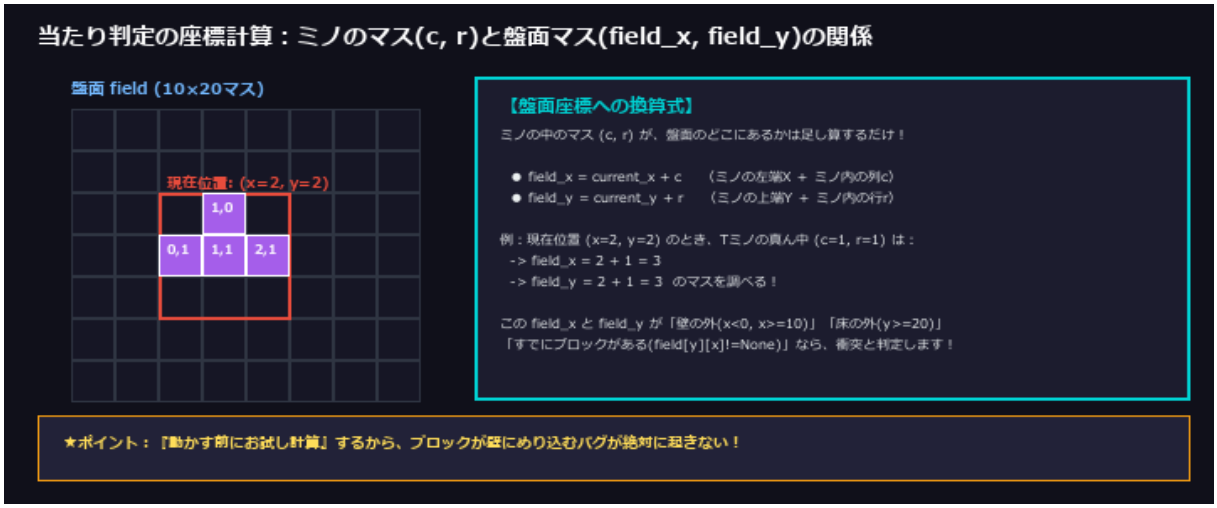
極意：「動かしてから戻すな、動かす前にお試しせよ！」

ブロックを動かした後に壁を突き抜けていたら戻す、というやり方はバグやチラつきの温床になります。
『一歩進める前に、次の場所が安全かをお試しチェックする』のがプロのゲーム開発の鉄則です！

【ポイント】 can_place がチェックする3大ルール

① 左右の壁: fx < 0 or fx >= COLUMNS ➔ アウト！
② 床（底）: fy >= ROWS ➔ アウト！
③ 既設ブロック: field[fy][fx] is not None ➔ アウト！
すべてクリアなら True（安全！）、1つでもダメなら False（移動キャンセル）！

ミノ座標から盤面座標への換算とコード解剖



can_place() 関数

```
def can_place(shape, offset_x, offset_y):
    for r in range(len(shape)):
        for c in range(len(shape[r])):
            if shape[r][c] == 1:
                fx = offset_x + c # 盤面のX = ミノのX + c
                fy = offset_y + r # 盤面のY = ミノのY + r

                # ① 左右の壁チェック
                if fx < 0 or fx >= COLUMNS:
                    return False
                # ② 底（床）チェック
                if fy >= ROWS:
                    return False
                # ③ すでに置かれたブロックチェック
                if fy >= 0 and field[fy][fx] is not None:
                    return False
    return True # 全ブロック安全なら置ける！
```

【ポイント】 短絡評価で IndexError を防ぐ知恵

先に fx < 0 or fx >= COLUMNS で弾いているため、field[fy][fx] に範囲外の数字が入ってクラッシュすることが絶対にありません！

第5回：くるっと回そう！回転の仕組み

【この回のゴール】 2次元配列の回転アルゴリズムを理解し、A/Dキーで回転させる



コンピュータの中で「回す」とはどういうこと？

パソコンに「画像を回転させる魔法のボタン」はありません。あるのは 0 と 1 の2次元リスト だけ！
つまり、『数字の並び順を書き換えて、90度回った新しいリストを作り出す』ことが回転の正体です。
7種×4方向＝28通りの形を手打ちする必要はありません。たった1つの数学ルールで回せます！

中学生でもわかる回転の数学！2ステップの魔法

折り紙のようにパタンパタンと2回折るだけで、完璧な90度右回転が完成します！



回転の2ステップ解説

・ステップ① 転置（Transpose）：斜め対角線でパタンと折り、横の行を縦の列に入れ替える！
・ステップ② 反転（Reverse）：各行を左右鏡のようにパタンとひっくり返す！
➔ すると、元の形がピッタリ時計回りに90度右回転した形へと早変わりします！

回転コードの実装とA/Dキーの作り分け

回転関数

```
def rotate_right(shape):
    # 2重for文: 元の (r, c) が、新しい (c, N-1-r) にワープする！
    N = len(shape)
    new_shape = [[0] * N for _ in range(N)]
    for r in range(N):
        for c in range(N):
            new_shape[c][N - 1 - r] = shape[r][c]
    return new_shape

def rotate_left(shape):
    # 左回転は「右回転を3回行う」のと全く同じ！
    return rotate_right(rotate_right(rotate_right(shape)))

def try_rotate(direction):
    global current_shape
    if current_type == "O": return # Oミノは回転不要
    new_s = rotate_right(current_shape) if direction == "right" else rotate_left(current_shape)
    if can_place(new_s, current_x, current_y):
        current_shape = new_s # 置けるなら回転確定！
```

【ポイント】 プロの1行必殺技 zip(*shape[::-1])

shape[::-1] で上下反転し、zip(*...) で転置することで、たった1行で右回転できます！

第6回：盤面固定・ライン消去・基本テトリス完成！

【この回のゴール】 底に着いたミノを盤面に焼き付け、動かないブロックに変える

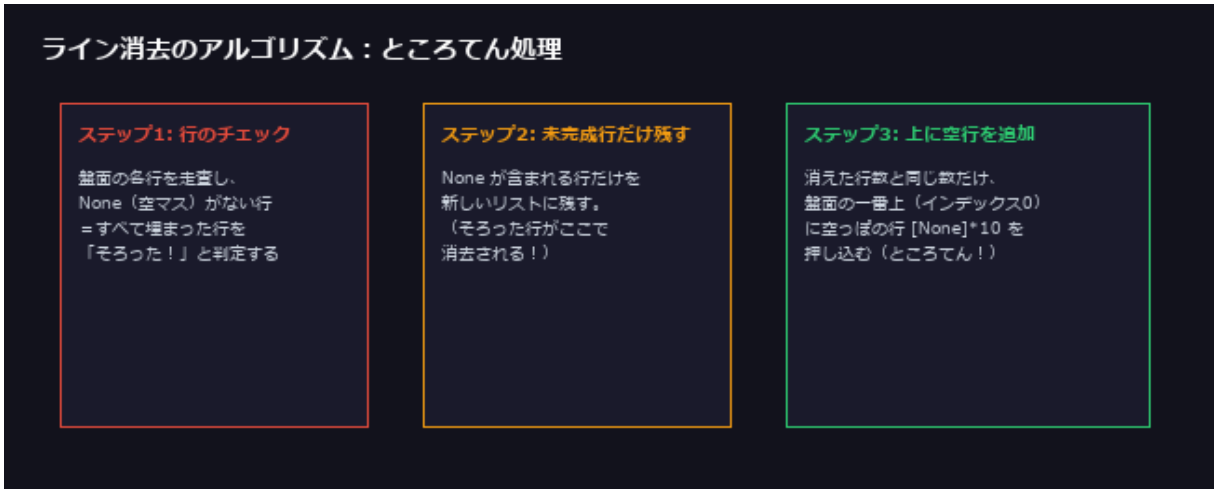


lock_piece()

```
def lock_piece():
    global current_shape, current_x, current_y, current_type
    color = COLORS[current_type]
    for r in range(len(current_shape)):
        for c in range(len(current_shape[r])):
            if current_shape[r][c] == 1:
                fx = current_x + c
                fy = current_y + r
                if 0 <= fy < ROWS and 0 <= fx < COLUMNS:
                    field[fy][fx] = color # 盤面に色を焼き付ける！

    clear_lines() # 揃った行があれば消去！
    spawn_new_piece() # 次の新しいミノを出す！
```

超気持ちいい「ライン消去」ところてん処理



clear_lines()

```
def clear_lines():
    global field
    # 1. 空マス(None)が1つでもある「未完成行」だけ残す！
    new_field = [row for row in field if None in row]
    lines_cleared = ROWS - len(new_field)

    # 2. 消えた行数分、一番上に新しい空行 [None]*10 を押し込む（ところてん！）
    for _ in range(lines_cleared):
        new_field.insert(0, [None for _ in range(COLUMNS)])

    field = new_field
```

新ミノ出現・ゲームオーバー・第1部完成！



spawn_new_piece()

```
def spawn_new_piece():
    global current_shape, current_type, current_x, current_y, is_game_over
    current_type = random.choice(list(SHAPES.keys()))
    current_shape = SHAPES[current_type]
    current_x = 3
    current_y = 0

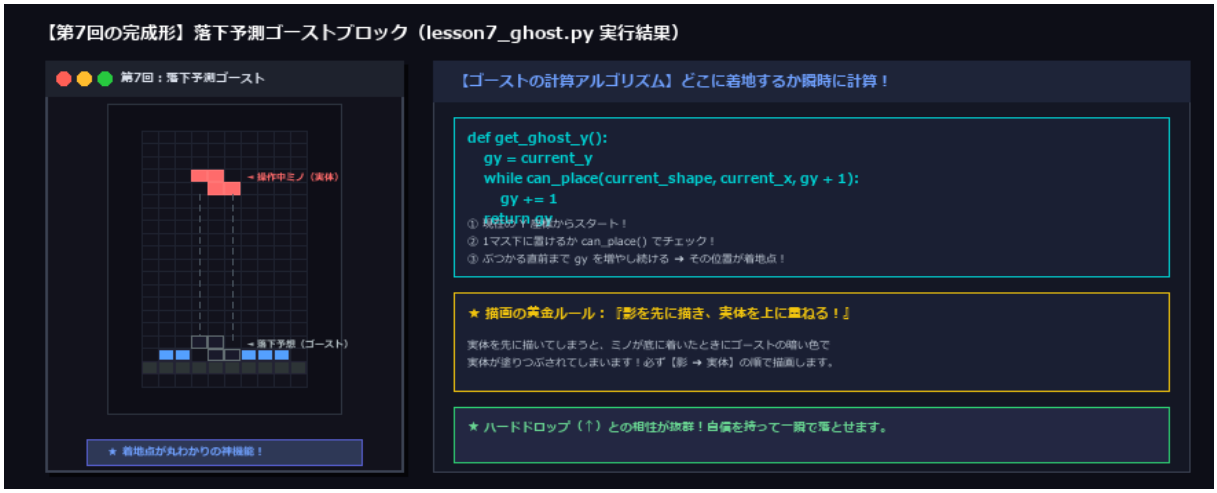
    # 出現した瞬間にもう置けない ➔ 天井到達（ゲームオーバー！）
    if not can_place(current_shape, current_x, current_y):
        is_game_over = True
```

【ポイント】 第1部完結！

★★★ 祝・第1部完成！完全テトリスが完成しました！ ★★★
lesson6_basic_tetris.py を実行してみましょう！
落下、移動、回転、ハードドロップ、固定、ライン消去、ゲームオーバー……100%遊べる本格テトリスが完成です！
第2部では、世界の公式テトリスと同じ神機能（ゴースト、7-bag、ロックディレイ、ホールド、SRS、T-Spin）を追加します！

第7回：落下予測！ゴーストブロック（影）の仕組み

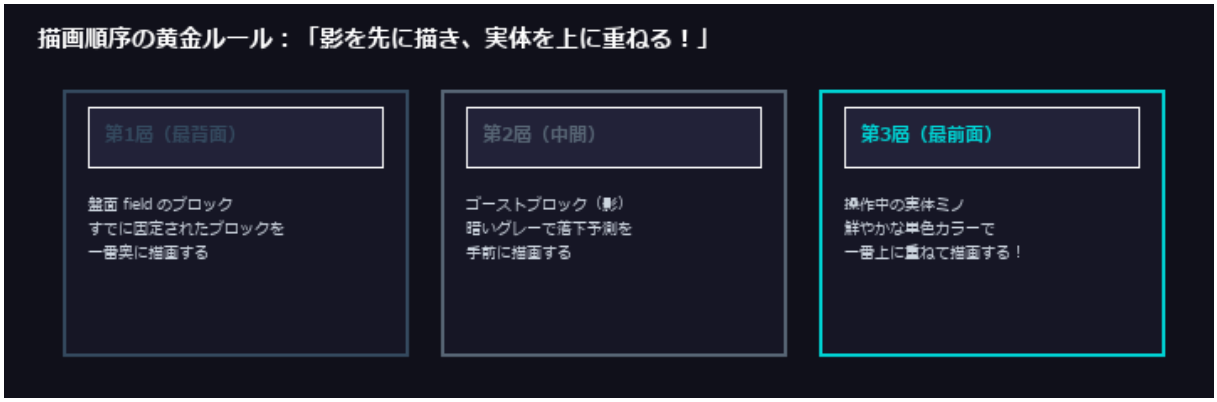
【この回のゴール】 着地予想地点に半透明・グレーの影をリアルタイム表示する



ゴースト位置の計算：限界まで落とし続けるwhileループ

現在の current_y からスタートし、can_place(..., gy + 1) が可能な限り下へ落とし続けます。
ぶつかって進めなくなった位置が、ゴーストブロックの着地座標になります！

ゴーストブロックの実装と描画の重ね順



ゴースト描画

```
def get_ghost_y():
    gy = current_y
    while can_place(current_shape, current_x, gy + 1):
        gy += 1
    return gy

def draw():
    canvas.delete("all")
    draw_field() # 1. 固定ブロックを描く

    # 2. ゴースト（影）を先に描く！（暗いグレー）
    gy = get_ghost_y()
    draw_piece_at(current_shape, current_x, gy, fill="#1a1a2b", outline="#4f5b66")

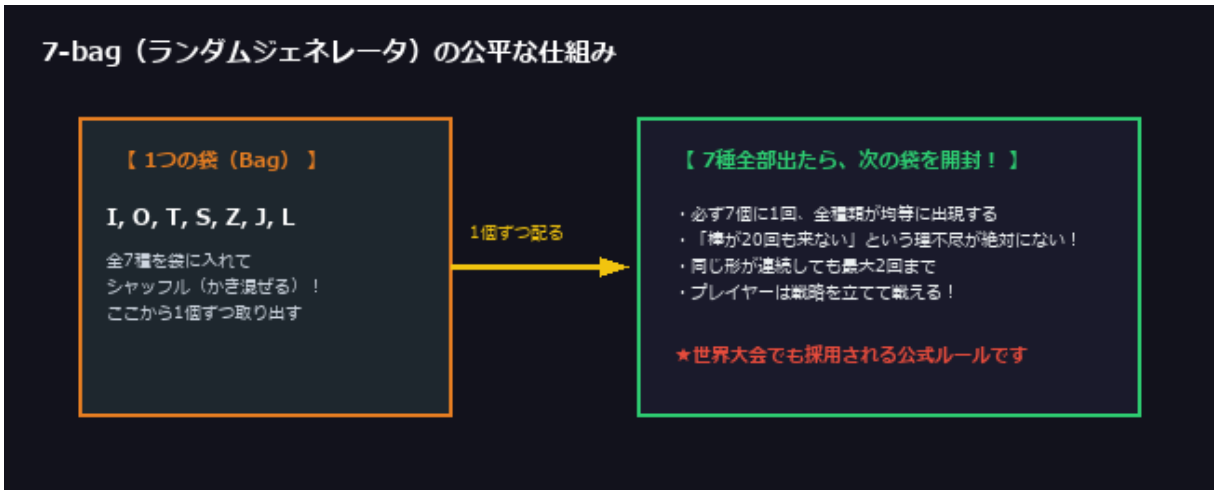
    # 3. 実体ミノを上に重ねて描く！（鮮やかな単色カラー）
    draw_piece_at(current_shape, current_x, current_y, fill=COLORS[current_type])
```

【ポイント】 描画の黄金ルール「影を先に描き、実体を上に重ねる！」

実体を先に描くと、着地時にゴーストの暗い色で実体が上書きされてしまいます！必ず影を先に描きましょう。

第8回：公平なランダム！「7-bag」システムの秘密

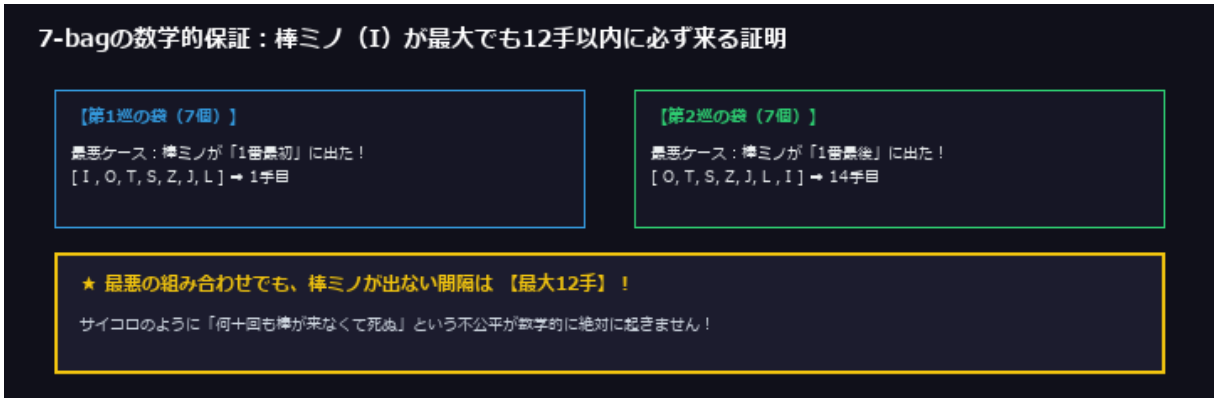
【この回のゴール】 公式テトリスの公平な抽選アルゴリズム「7種1巡」を実装する



サイコロ乱数の罠 & 公式「紙袋シャッフル」システム

サイコロのように random.choice() で選ぶと、同じミノが連続したり棒ミノが何分も来ない「ミノ枯渇」が起きます。
公式テトリスでは、7種類のミノを紙袋に入れてシャッフルし、1個ずつ取り出す「7-bag」を採用しています！

7-bagの実装と確率の数学



7-bag 抽選関数

```
import random
bag = []

def get_next_piece():
    global bag
    # 袋が空っぽになったら、新しく7種を入れてシャッフル！
    if len(bag) == 0:
        bag = list(SHAPES.keys()) # ['I', 'O', 'T', 'S', 'Z', 'J', 'L']
        random.shuffle(bag) # ガラガラと混ぜる！

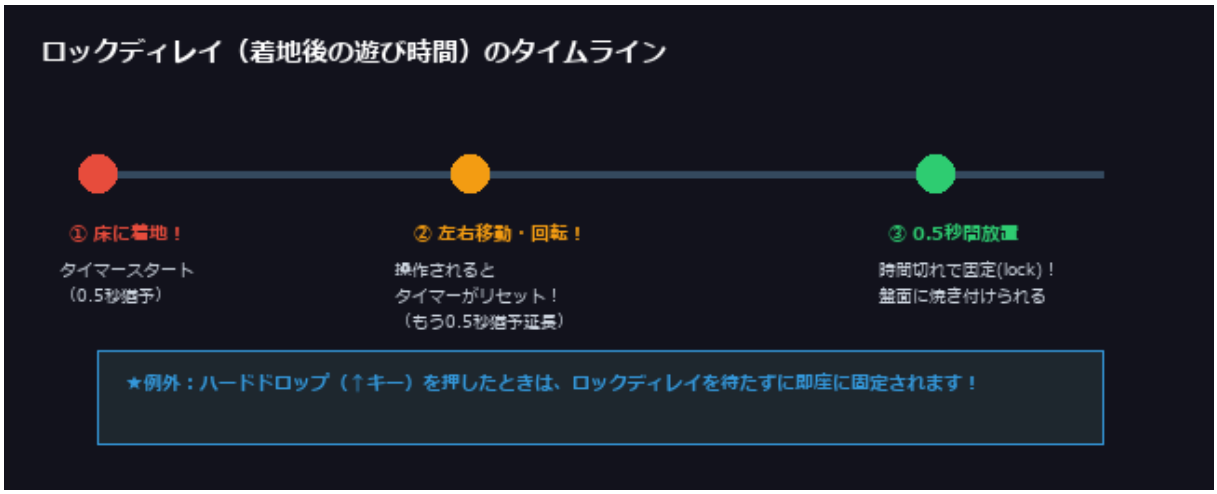
    return bag.pop(0) # 先頭から1つ取り出す！
```

【ポイント】 棒ミノ（I）が出ない間隔は【最大でも12手】！

第1巡で最初に出て、第2巡で最後に出る最悪ケースでも、最大12手以内に必ず棒が来ます！プレイヤーは戦略を立てられます。

第9回：ギリギリ操作！ロックディレイ（遊び時間）

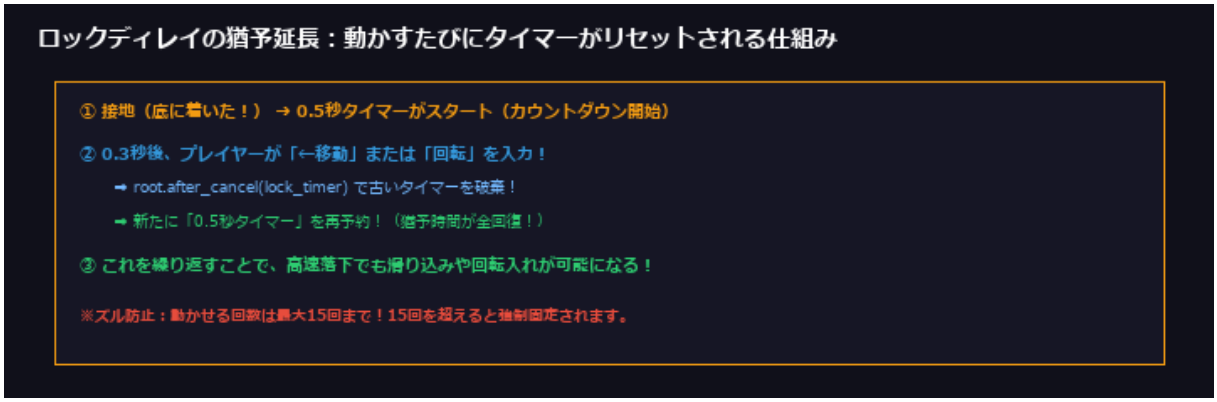
【この回のゴール】 底に着いてから約0.5秒間の猶予時間（ロックディレイ）を作る



着地即固定をなくし、神プレイを可能にする「0.5秒の遊び時間」

底に着いた瞬間に即固定されると、横への滑り込み（スライド）や回転入れが一切できず操作不能になります。
公式ルールでは、底に着いても約0.5秒（500ms）の猶予が与えられ、動かすたびに猶予が延長されます！

タイマーIDキャンセルとリセットの実装



ロックディレイ

```
lock_timer = None
lock_reset_count = 0

def check_ground():
    global lock_timer
    if not can_place(current_shape, current_x, current_y + 1):
        if lock_timer is None:
            lock_timer = root.after(500, do_lock) # 0.5秒後に固定予約
    else:
        cancel_lock_timer() # 浮いたら解除

def on_move_or_rotate():
    global lock_reset_count
    if not can_place(current_shape, current_x, current_y + 1):
        if lock_reset_count < 15: # 最大15回までリセット可能
            cancel_lock_timer()
            lock_reset_count += 1
            check_ground() # タイマー再スタート（猶予延長！）
```

第10回：ホールド機能＆NEXTミノ表示

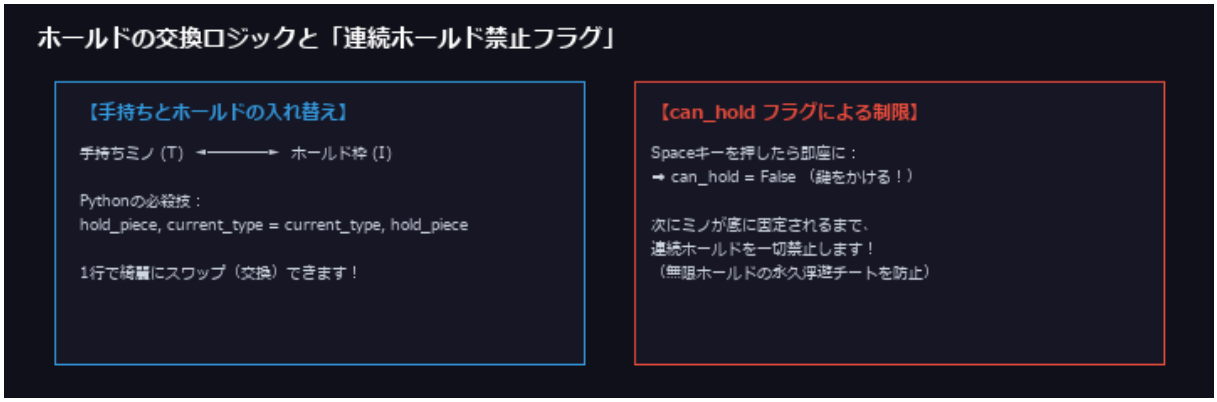
【この回のゴール】 Spaceキーでミノをキープ・交換するホールド機能を実装する



ホールドの2分岐：初回キープ vs 2回目以降のスワップ交換

・ホールド枠が空のとき: 今のミノを格納し、NEXTから新しいミノを出撃させる。
・すでにホールドがあるとき: 今のミノとホールドミノをスワップ（交換）する！

連続ホールド禁止フラグとNEXT枠描画



hold() 関数

```
hold_piece = None
can_hold = True # 1手1回制限フラグ

def hold():
    global current_type, current_shape, hold_piece, can_hold, current_x, current_y
    if not can_hold: return # すでに交換済みなら無視！

    if hold_piece is None:
        hold_piece = current_type
        spawn_new_piece()
    else:
        # 1行でスワップ（交換！）
        hold_piece, current_type = current_type, hold_piece
        current_shape = SHAPES[current_type]
        current_x, current_y = 3, 0

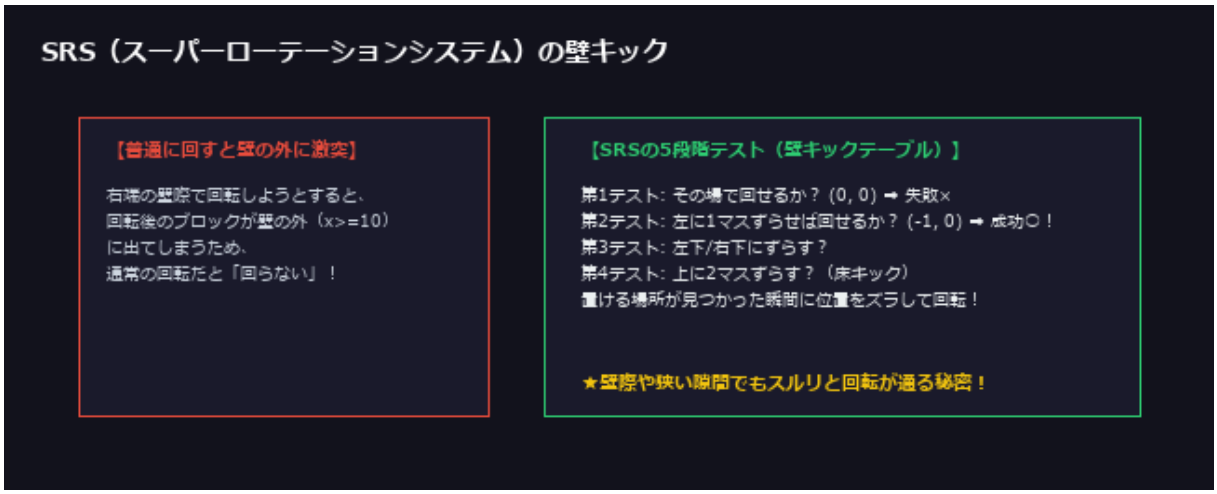
    can_hold = False # 固定されるまで再ホールド禁止！
```

【ポイント】 can_hold がないとどうなる？（無限ホールドチート）

Spaceキー連打で永久にミノが着地しなくなります！lock_piece() で can_hold = True に戻します。

第11回：壁キックの奇跡！公式回転「SRS」

【この回のゴール】 壁際や狭い隙間でもスッと回る「SRS壁キック」を実装する



「回転詰まり」を解消するSRS（スーパーローテーションシステム）

壁にピッタリくっついていると、回転後の形が壁の外にはみ出るため普通の回転では回らなくなります。
公式テトリスでは、「壁を蹴って（キック！）少しズラした位置に入らないか試す」ことでスパスパ回ります！

SRSの5段階テストと右端壁キックの実例

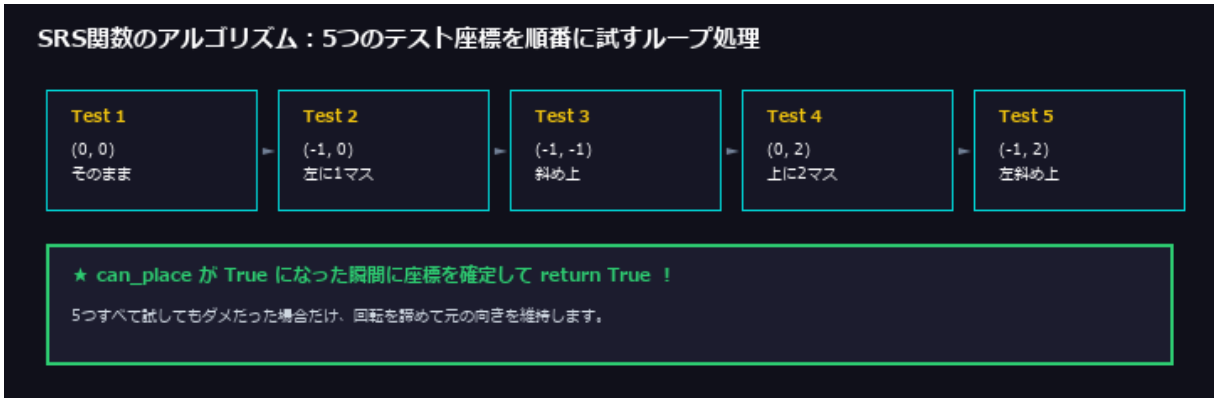
壁際でミノを回そうとしたとき、プログラムがどうやって回避場所を探すかを図解します。



SRSの5段階オフセットテスト

・テスト1 (0, 0): その場で回す ➔ 壁に激突して失敗×
・テスト2 (-1, 0): 左に1マスずらす ➔ 成功○！ここに決定！
・テスト3: 斜め上 (-1, -1) ｜ テスト4: 上に2マス (0, 2) ｜ テスト5: 左上 (-1, 2)

SRS回転関数の実装とプロ仕様の操作感



SRS 回転関数

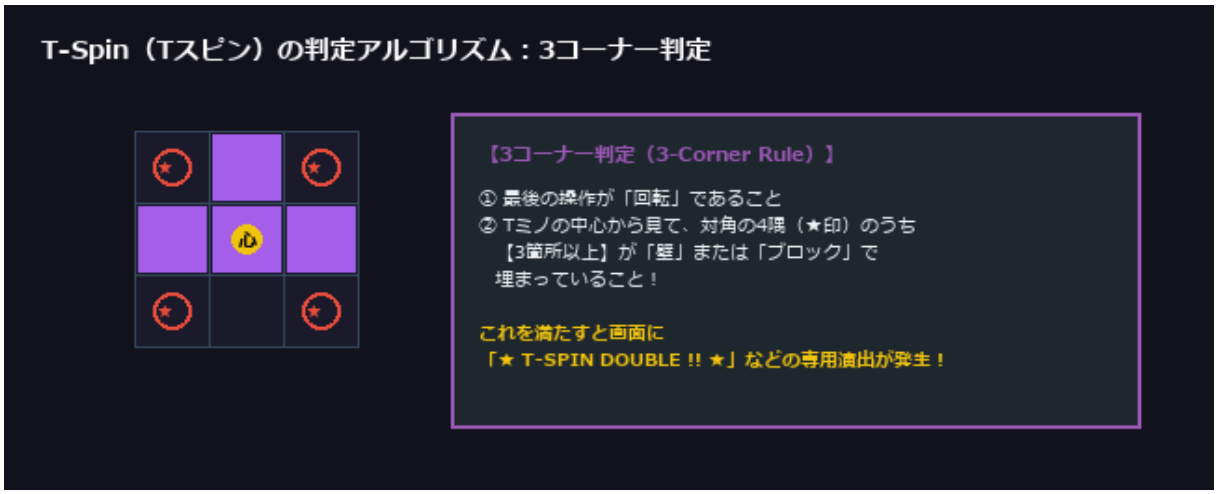
```
SRS_KICKS = [(0, 0), (-1, 0), (-1, -1), (0, 2), (-1, 2)]

def try_rotate_srs(direction):
    global current_shape, current_x, current_y
    if current_type == "O": return
    new_s = rotate_right(current_shape) if direction == "right" else rotate_left(current_shape)

    # 5つのテスト位置を順番に試し、入る場所をスッと見つける！
    for dx, dy in SRS_KICKS:
        if can_place(new_s, current_x + dx, current_y - dy):
            current_shape = new_s
            current_x += dx
            current_y -= dy
            return True
    return False
```

第12回：T-Spin判定&テトリス究極完成！

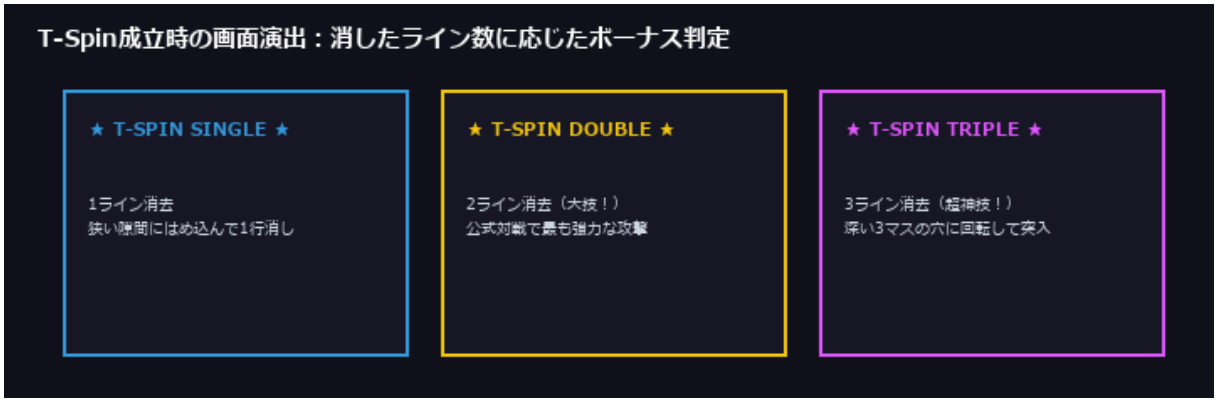
【この回のゴール】 プロゲーマーの必須テクニック「Tスピン」を判定し演出しよう！



プロの神技「T-Spin」を見分ける【3コーナー判定】の幾何学

Tミノの中心から見て、対角の4隅（★印）のうち【3箇所以上】が壁やブロックで塞がれているなら、「回転させなければ絶対にその場所に収まらなかった」ことの完全な幾何学的証明になります！

T-Spin判定の実装コードとボーナス演出



T-Spin 判定ロジック

```
def check_t_spin():
    if current_type != "T" or not last_action_was_rotate:
        return False

    cx, cy = current_x + 1, current_y + 1
    corners = [(cx-1, cy-1), (cx+1, cy-1), (cx-1, cy+1), (cx+1, cy+1)]

    # 4隅中、壁・床・ブロックで塞がっている角をカウント
    occupied = sum(
        1 for x, y in corners
        if x < 0 or x >= COLUMNS or y >= ROWS or (y >= 0 and field[y][x] is not None)
    )
    return occupied >= 3 # 3箇所以上塞がっていれば T-Spin 成立！
```

究極完成！完全アーキテクチャと修了の証



【ポイント】 コース修了おめでとうございます！

□ 【公式仕様テトリス完全攻略マスター認定証】 □

おめでとうございます！ あなたは画面描画の基本から始め、2次元配列のデータ管理、ゲームループ、当たり判定、
回転の幾何学、ゴースト、公平乱数、ロックディレイ、SRS壁キック、そしてT-Spin判定に至るまで、
世界の公式テトリスを構成するすべての高度なアルゴリズムを自力で完全実装しました！

この教材で学んだ「お試し一歩の当たり判定」「二次元配列の転置と反転」「イベント駆動のゲームループ」は、
あらゆるゲーム・アプリ開発で一生通用する本物のプログラミング力です。自信を持って次の挑戦へ進んでください！